CIVITA: arquitectura y relaciones UML corregidas

La implementación tiene un **cliente Android de despliegue monolítico**, organizado internamente con **MVVM por capas, patrón Repository e inyección manual de dependencias**. Firebase actúa como **Backend as a Service (BaaS) externo**. Son clasificaciones de dimensiones diferentes: unidad de despliegue, organización interna y servicios remotos.

Parte	Implementación	Responsabilidad
View	Funciones Jetpack Compose en ui/screens y CivitaApp	Representar el estado, capturar acciones y navegar.
ViewModel	AuthViewModel, CivitaViewModel, CommunityViewModel	Validación y coordinación, corrutinas, sesión y estado observable StateFlow.
Model / datos	Tipos de model y repositorios de data	Datos tipados, acceso a Firebase, listeners y transacciones.
Contratos	ReportRepository y CommunityActions	Operaciones que requieren el coordinador y las pantallas, respectivamente.
Composición	CivitaApp, AppContainer y las fábricas de ViewModels	Construcción y suministro manual de dependencias.
Backend administrado	Firebase Authentication, Cloud Firestore y Cloud Storage	Identidad, documentos y archivos; autorización mediante reglas.

La denominación se sustenta en CivitaApp.kt: Compose observa los estados y entrega acciones a ViewModels; éstos llaman a repositorios que encapsulan SDK Firebase. AppContainer.kt construye los repositorios. Las fábricas construyen los ViewModels y Compose los liga al ciclo de vida y al UID. No hay un framework de inyección de dependencias ni una capa separada de casos de uso.

Dos entregables principales

• **CIVITA-Clases.pdf:** una sola página horizontal panorámica (1189 × 620 mm), vectorial. Incluye 25 clasificadores principales de las funcionalidades y 30 relaciones explícitas. La tipografía conserva detalle al ampliar. No reducir a A4 para revisar firmas.

• **CIVITA-Componentes.pdf:** una sola página horizontal panorámica (841 × 418 mm), vectorial.

• **CIVITA-Diagramas.pdf:** recopilación de los dos anteriores, exactamente dos páginas: una por diagrama.

La lámina principal de clases selecciona autenticación, publicación, comunidad, estados, configuración y las entidades principales. Los tipos de detalle referenciados en las firmas (validaciones, apoyos, evidencias, eventos, capas, etc.), las fábricas anidadas y las extensiones privadas se documentan en CATALOGO-CODIGO.md y las vistas de detalle conservadas. No se afirma que 25 clasificadores constituyan un inventario de todas las clases del repositorio.

Generalizaciones y realizaciones efectivas

Relación	Tipo UML	Línea y extremo	Evidencia
AuthViewModel → ViewModel	Generalización	Continua, triángulo hueco hacia ViewModel	ui/AuthViewModel.kt
CivitaViewModel → ViewModel	Generalización	Continua, triángulo hueco hacia ViewModel	ui/CivitaViewModel.kt
CommunityViewModel → ViewModel	Generalización	Continua, triángulo hueco hacia ViewModel	ui/CommunityViewModel.kt
MainActivity → ComponentActivity	Generalización	Continua, triángulo hueco hacia ComponentActivity	MainActivity.kt
FirebaseReportRepository → ReportRepository	Realización	Discontinua, triángulo hueco hacia la interfaz	data/FirebaseReportRepository.kt
CommunityViewModel → CommunityActions	Realización	Discontinua, triángulo hueco hacia la interfaz	ui/CommunityViewModel.kt

Las tres clases anidadas Factory también realizan ViewModelProvider.Factory; constan en el catálogo y la vista de fábricas, fuera del alcance principal de esta lámina. Las entidades Kotlin data class no heredan entre sí.

Asociaciones, multiplicidad y dependencias

• Cada ViewModel conserva exactamente un repositorio de su tipo. Puede haber cero o más ViewModels referenciando el mismo repositorio. Asociación navegable: línea continua con flecha abierta; 1 en el repositorio y 0..* en el otro extremo.

• AppContainer expone repository, authRepository y communityRepository como referencias opcionales (0..1), pues la configuración Firebase puede fallar. No se afirma propiedad exclusiva de SDK singleton.

• Los estados conservan referencias opcionales, únicas o listas: profile/config = 0..1, preferences = 1, listas de reportes/categorías/comentarios = 0..*. La relación state.value es derivada de StateFlow; no agrega una propiedad nueva al código.

• UrbanReport.status referencia un ReportStatus (1).

• Las líneas verdes son **asociaciones lógicas del dominio**: categoría–reporte mediante categoryId, autor–reporte mediante creatorId, autor–comentario mediante authorId, reporte–comentario mediante la ruta Firestore y usuario–preferencias mediante su ruta. No representan campos de objetos anidados que el código no tiene, ni claves foráneas SQL.

• Las flechas discontinuas abiertas representan uso de un tipo, parámetro o servicio: ReportDraft, AuthValidation, GeoHash y servicios en el diagrama de componentes. Las dependencias no llevan cardinalidades.

• No se usan rombos de agregación o composición: el código y Firestore no acreditan exclusividad del objeto ni destrucción en cascada. Una subcolección de Firestore no se elimina automáticamente con su documento padre.

Componentes

Los rectángulos con icono de componente representan módulos/responsabilidades. Los círculos representan interfaces proporcionadas: ReportRepository por los datos y CommunityActions por la coordinación. El consumidor depende de la interfaz mediante una flecha discontinua abierta. Los archivos de reglas se

muestran como artefactos, no como clases ni componentes de negocio.

Las dependencias remotas van desde los SDK del cliente hacia Authentication, Firestore y Storage. Credential Manager obtiene una credencial Google y Firebase Auth la intercambia por una sesión. Las reglas Storage consultan al autor y contador en Firestore; esa dependencia también se representa.

Notación y límites

UML 2.5.1: https://www.omg.org/spec/UML/2.5.1/ . Las firmas conservan tipos Kotlin (String, Int, Long, Unit, Flow, StateFlow), visibilidad y retornos. ? identifica nulabilidad; {readOnly} propiedades val; {suspend} documenta la semántica Kotlin de una operación suspendida. Las data classes generan sus operaciones estándar, indicadas en la leyenda.

La disposición editorial queda en los SVG y los generadores Python incluidos. Los .puml conservan el modelo semántico editable; PlantUML puede redistribuirlo al regenerarlo. Los diagramas describen la implementación, no certifican todos los recorridos Android. La salida Cloud Shell compartida por el usuario confirmó sesión Firebase de cliente, publicación, consulta y limpieza; Google Android y Storage mantienen su validación independiente.

Clasificación de despliegue y respuesta para la sustentación

settings.gradle.kts declara únicamente include(":app"): el código Android se compila y distribuye como una APK. Los componentes dibujados dentro de ese límite son responsabilidades lógicas; no son aplicaciones ni microservicios desplegables de forma independiente. La denominación precisa es **cliente monolítico Android organizado con MVVM por capas, conectado a Firebase BaaS externo**.

Frase sugerida: «Nuestra aplicación Android se despliega como un monolito, una sola APK. Internamente usamos MVVM y repositorios para separar la interfaz, la coordinación y los datos. Authentication, Firestore y Storage son servicios externos administrados por Firebase». MVVM por sí solo no determina si un despliegue es monolítico.

Interfaz de usuario explícita en componentes

El bloque compuesto **Interfaz de usuario / View** contiene AuthScreen, MapScreen, ReportScreen, CommunityScreen, ReportDetailScreen y el grupo de cuenta con ProfileScreen y PreferencesScreen: siete funciones reales de pantalla. El actor Usuario interactúa con ese bloque, que depende de los ViewModels y de sus acciones. El límite exterior indica **una APK / módulo :app**. Las funciones Compose se representan como componentes lógicos de presentación; no se inventan clases Kotlin para ellas.

Los círculos **I1 = CommunityActions** e **I2 = ReportRepository** representan interfaces de programación UML proporcionadas por coordinación y datos. Se distinguen expresamente de la interfaz visual de usuario.

Formato horizontal y comprobaciones

Clases: horizontal panorámica (1189 × 620 mm), una página; componentes: horizontal panorámica (841 × 418 mm), una página. La recopilación tiene exactamente dos páginas horizontales. El diagrama de clases conserva los mismos 25 clasificadores y 30 relaciones; se redistribuyó en cinco columnas y los conectores se encaminan por corredores libres sin atravesar las cajas. PDF y SVG conservan texto y líneas vectoriales para ampliar.

Explicaciones separadas de los diagramas

Los PDF de diagramas contienen sólo el modelo UML, sus títulos y las etiquetas necesarias. Los textos sobre arquitectura, notación, alcance y sustentación se entregan en documentos independientes. I1 =

CommunityActions; I2 = ReportRepository. Las láminas mantienen una página horizontal cada una, con el lienzo ajustado al contenido.